assignment 5

Live Data System

Design, Build, Ship · MPCS 51238 · Autumn 2026

Due Tuesday, November 3, at 5:30 PM

Overview

Last week you built a full-stack app. This week you'll add a worker that runs every few seconds, around the clock, fetching from a live data source and saving to your database, with a page that updates on its own. Extend Assignment 4 or start something new. Your classmates should be able to sign in, choose what they follow, and watch it change.



This assignment is also a learning journey. Each step ends with an Ask Your Agent prompt, so your agent teaches you what you just built: it explains it, draws it, or quizzes you on it.

A Heads Up

This assignment is more complex than previous assignments: a full system, with multiple services running on different platforms and talking to each other. Start early, and budget more time than you think you need. Reach out to any of the teaching staff if you need help.

> ABOUT RAILWAY
>
> Railway had a data breach in October and is currently preventing new signups, so we'll use Supabase for the worker. If you're able to sign up for Railway, go ahead.

Steps

1. Choose a Live Source

Pick a source that changes on its own while people are watching. All of these work without an API key:

- Chicago bikes: Divvy
- Earthquakes: USGS
- Live game scores: ESPN
- Wikipedia edits: Wikimedia
- Flights near O'Hare: OpenSky
- The space station: Where the ISS At?
- City requests: Chicago 311
- Boston trains: MBTA

Or use your own idea: if it polls, stores, and pushes updates, it counts. Before you build on it, have your agent call it twice, a few minutes apart, and show you what changed.

ASK YOUR AGENT

“Explain this response: what each field means, how often it changes, and which fields my app needs.”

2. Give Your Agent Access

Connect your agent to each part of your system, so it can set things up and check its own work:

- **Supabase's MCP server,** scoped to your project, as in Assignment 4: the tables, Realtime, the logs, and the worker. Your agent deploys the worker and sets its schedule through it, so if you made the server read-only in Assignment 4, turn that off for this one.
- **Clerk's CLI,** `clerk`**:** sign-in and your users, as in Assignment 4.
- **Vercel:** as before, it deploys your page when you push to GitHub.

ASK YOUR AGENT

“List the MCP servers and CLIs you can use in this project, and what you'd use each one for. Then check that each one works.”

3. Plan the Data Flow

Plan with your agent how data moves through your system: from the source, to the worker, to the database, to the page. Have Claude draw it as a high-level chart before any code is written.

ASK YOUR AGENT

“Walk me through the chart: what each part sends to the next, and what would break if one part changed it.”

4. Build the Worker

The worker is a Supabase Edge Function: a small TypeScript function that runs on Supabase's servers, with its code in your repository's `supabase/functions` folder. Supabase Cron calls it every few seconds, or less often if your source changes slowly or limits how often you can call it. Each time, it fetches the source and saves only the rows that changed, with an upsert, which inserts new rows and updates changed ones. It isn't a user, so it writes with your Supabase secret key. Supabase gives the function that key on its own servers, and it never goes in your code. Each time it runs, it adds a row to a table of runs: when it ran, whether it worked, and how many rows changed. Have your agent deploy it and call it once, and watch the rows change. Then schedule it with Supabase Cron, and have your agent set up a test that runs the worker on a saved copy of the source's response and checks what it saves, and make sure you understand what it checks.

ASK YOUR AGENT

“Show me the worker's logs from its last three runs, and explain each line.”

5. Make the Page Live

The page updates without a refresh, using Supabase Realtime, which pushes each change in the database to the open page. Realtime needs three things: the table added to Realtime, read access granted, and a row-level security policy that lets the person read it. If one is missing, the page gets nothing, with no error. Each person

chooses what they follow, and sees only their own choices. Show how fresh the data is, and design each state: loading; fresh; stale, when the data is older than a few missed runs; and down, when the worker's last run failed.

> ASK YOUR AGENT
>
> “Trace one update for me, from the source to my screen, and name every part it passes through.”

6. Deploy & Test

Deploy the page to Vercel, with your environment variables set there, and check that the worker is running. Have a classmate sign in, choose what they follow, and watch it update.

> ASK YOUR AGENT
>
> “Quiz me on how my system works, one question at a time, and tell me what I got wrong. Start with the questions in Be Ready to Explain.”

Requirements

- **Live source:** data from one source that changes on its own.
- **Data flow:** a high-level chart of how data moves through your system, made before the code.
- **Worker:** an Edge Function that Supabase Cron runs every few seconds, or a worker on Railway; it saves only what changed and records each run in a table.
- **Live page:** updates without a refresh, shows how fresh the data is, and has loading, stale, and down states.
- **Personal:** each person chooses what they follow, with row-level security on their choices.
- **Deployed:** the page live on Vercel and the worker running, where a classmate can sign in and watch it update.
- **UI/UX:** remember the design process, and the UI/UX design principles and flows we've discussed. Live data needs clear states.

Tech Stack

Next.js and Tailwind CSS, Clerk for sign-in, and Supabase for the database, Realtime, and the worker: an Edge Function that Supabase Cron runs every few seconds. Edge Functions run on Deno instead of Node.js, so add `supabase` to the `exclude` list in `tsconfig.json`, or your Next.js build will check the worker's code and fail. Put the code in your own GitHub repository, deploy the page on Vercel's free Hobby plan, and deploy the worker to Supabase.

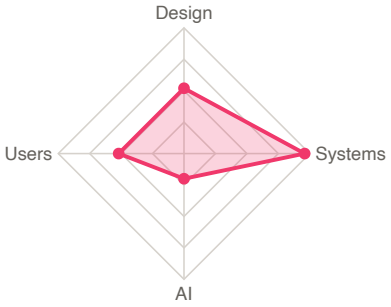
Your project's context file is `AGENTS.md`. If there's a `CLAUDE.md`, move anything useful into `AGENTS.md` and delete it, then check with `/context` that `AGENTS.md` loaded.

Submission

Submit in Google Classroom:

1. **Vercel URL:** a live link to your app.
2. **GitHub URL:** your repository, with your `README.md` and a commit history that shows your progress.

Assessment



This Assignment's Focus: Systems

Most of the work is in the system: the worker, the database, and updates in real time. Design and users are part of it too: the page's states, and people who sign in and follow what they choose.

A complete assignment has:

- A live Vercel URL where a classmate can sign in and watch the data update without refreshing
- A worker that runs every few seconds on Supabase or Railway, recording each run in a table
- A high-level chart of your data flow
- Freshness shown on the page, with a designed stale state
- A GitHub repository with commits that show your progress
- You're ready to explain your work in class (below)

A one-shot submission, made by giving this brief to an agent once and turning in the result, is marked incomplete.

Be Ready to Explain

In class, you'll talk through your work. Be ready to explain:

1. **Tracing a data change:** when the data source changes, what happens in the worker, in the database, and on someone's screen?
2. **The data flow:** how data moves from the source to the page, and what would break if one part changed.
3. **Freshness:** how someone knows the data is current.
4. **Agentic workflow:** what your agent's tests check, and what you found in the logs.

Free Tiers and Usage Limits

Aside from Claude Code, you shouldn't need to pay for anything in this course. For every service you sign up for, such as Vercel, Supabase, Clerk, Railway, or a model API:

- Stay on the free tier.
- Set usage limits and spending alerts wherever the service offers them.
- Avoid adding a payment method unless a service requires one. If it does, set a spending cap first.

Supabase's free plan includes 500,000 Edge Function calls a month. A worker that runs every 10 seconds uses about half of them, and one that runs every 5 seconds uses all of them, so every 10 seconds is a good place to start.

Free tiers have limits, too. Supabase Realtime counts every change sent to every open page, so a worker that rewrites every row every few seconds can use up a month of messages in a few hours; save only what changed. If your app makes too many requests, a provider can slow it down, refuse requests for a while, pause your project, or block your key or account. Supabase pauses a free project after a week without use, so open your app once in a while until grades are back.

ASK YOUR AGENT

“Walk me through the free tier limits for each service my app uses, and how to set usage limits and spending alerts for each one.”